

学校代码: 10246
学 号: 22302010001

復旦大學

本 科 毕 业 论 文

基于小样本学习的机器人决策规划技术研究

Research on Robot Decision Planning Technology Based on
Few-Shot Learning

院 系: 计算与智能创新学院

专 业: 软件工程

姓 名: 任立德

指 导 教 师: 戈维峰 副教授

完 成 日 期: 2026 年 5 月 26 日

目 录

摘要	iii
Abstract	iv
第 1 章 引言	1
1.1 研究背景	1
1.2 本文工作	1
1.3 关键术语与问题边界	2
1.4 论文结构	3
第 2 章 相关工作	4
2.1 视觉语言模型	4
2.2 思维链推理	4
2.3 Code2Logic	5
2.4 三维场景图	6
2.5 合成数据与域随机化	6
2.6 本章小结	7
第 3 章 方法	8
3.1 整体架构	8
3.2 Concept-Graphs 标注管线	8
3.2.1 六步管线	8
3.2.2 Prompt 设计	9
3.2.3 GPU 加速	9
3.3 Isaac Sim 验证系统	10
3.3.1 场景生成	10
3.3.2 真值导出	10
3.4 Caption 质量过滤	10
3.5 统一数据格式	11
第 4 章 实验	12
4.1 实验设置	12
4.1.1 硬件与软件	12
4.1.2 数据	12

4.2	游戏数据合成结果	12
4.3	真实场景标注结果	12
4.3.1	各场景产出	12
4.3.2	描述优化效果	13
4.3.3	瓶颈分析	14
4.4	Prompt 消融实验	14
4.5	合成场景验证	15
4.6	推理模型选型	15
4.6.1	基准测试	15
4.6.2	实际管线对比	16
4.7	新旧 Prompt 同场景对比	16
4.8	管线效率	17
4.9	三源数据对比	17
4.10	端到端验证	18
4.11	综合讨论	18
第 5 章 总结与展望		20
5.1	工作总结	20
5.2	局限性	21
5.3	未来方向	21
参考文献		23
致谢		25
完整 Prompt 模板		26
QA 数据 JSON Schema		28

摘 要

三维场景理解任务面临高质量多模态标注数据稀缺的核心困境。人工标注成本高昂——单个室内场景（约 70 个物体）的完整标注可能需要数小时的专家劳动——导致标注数据的规模和多样性严重受限。现有自动生成方法虽然降低了成本，但在关系推理精度和跨场景泛化性上存在明显不足：视觉语言模型倾向于生成光照、氛围等场景级感官描述，而非精确的物体类别和空间关系信息。

针对上述问题，本文提出了一种融合游戏逻辑驱动合成、真实场景理解和仿真真值验证的三层数据自动标注方法。首先，将 Code2Logic 的模板化数据合成范式从二维游戏领域迁移至三维场景，以 Concept-Graphs 为框架整合 GradSLAM、Grounded-SAM、CF-SLAM、LLaVA v1.5 和 GPU 加速的大语言模型推理，构建了从 RGB-D 图像序列到结构化 QA 数据集的端到端六步自动处理管线。通过专门设计的结构化 prompt，将物体类别识别率从接近 0 提升至约 85%。其次，针对 Ollama 推理引擎存在的 CUDA 版本不兼容问题，通过 llama-cpp-python 源码编译方案将推理吞吐量从 3 tok/s 提升至 99 tok/s（33 倍提升），单场景 LLM 处理时间从 15 分钟降至 50 秒。最后，集成 NVIDIA Isaac Sim 仿真环境构建了合成真值验证系统，通过程序化场景生成和几何规则计算，实现了标注精度的可量化评估。

实验结果表明，本文方法在三类数据源（30 款 2D 游戏、7 个真实室内场景、20 个合成仿真场景）上累计产出 2395 条结构化 QA 数据，覆盖 Target Perception、Spatial Reasoning、Scene Comprehension、State Prediction 和 Strategy Optimization 共 5 种推理类型。跨源对比评估揭示了合成真值对推断标注的可量化校准能力——Isaac Sim 基于几何规则的空间关系准确率为 100%，而 Concept-Graphs 依赖模型推断的准确率约为 70%。5 款国内开源大语言模型的基准测试表明，Qwen2.5-3B 在速度和质量的平衡上为最优选择（86 tok/s，100% 格式合法率）。

本研究为三维场景的自动化标注提供了一套低成本、可扩展、精度可验证的完整解决方案，对视觉语言模型训练数据的构建和多模态场景理解系统的开发具有一定的理论价值和工程参考意义。

关键词：三维场景理解；数据合成；视觉问答；游戏逻辑驱动；仿真验证

中图分类号：TP391

Abstract

Three-dimensional scene understanding faces the fundamental challenge of scarce high-quality multimodal annotation data. Manual annotation is prohibitively expensive—a single indoor scene with approximately 70 objects can require hours of expert labor—severely limiting the scale and diversity of training data for vision-language models. Existing automatic methods reduce cost but exhibit significant shortcomings in relational reasoning accuracy and cross-scene generalization. Specifically, current VLMs tend to generate scene-level sensory descriptions (e.g., lighting conditions, atmospheric qualities) rather than precise object-level semantic information.

To address these challenges, this thesis proposes a three-tier data annotation framework that integrates game-logic-driven synthesis, real-world scene understanding, and simulation-based ground-truth verification. The framework begins by migrating the template-based data synthesis paradigm of Code2Logic from 2D games to 3D scenes. Using Concept-Graphs as the integration framework, we construct an end-to-end six-step pipeline that combines GradSLAM (3D reconstruction), Grounded-SAM (2D segmentation), CF-SLAM (object-level mapping), LLaVA v1.5 (multimodal description), and GPU-accelerated LLM inference (description refinement and relation reasoning). A specially designed structured prompt increases the object category recognition rate from near zero to approximately 85%. Furthermore, we identify and resolve a CUDA version incompatibility in the Ollama inference engine by source-compiling llama-cpp-python, achieving a 33 $\times$ throughput improvement (3 $\rightarrow$ 99 tok/s) and reducing per-scene LLM processing time from 15 minutes to 50 seconds. Finally, we integrate NVIDIA Isaac Sim to construct a synthetic ground-truth verification system, enabling quantifiable evaluation of annotation accuracy through procedural scene generation and geometric rule computation.

Experimental results demonstrate that our method produces 2,395 structured QA pairs across three data sources (30 2D games, 7 real indoor scenes, and 20 synthetic simulation scenes), covering 5 reasoning types: Target Perception, Spatial Reasoning, Scene Comprehension, State Prediction, and Strategy Optimization. Cross-source comparative evaluation reveals the quantifiable calibration capability of synthetic ground truth—Isaac Sim achieves 100% spatial relation accuracy via geometric rules, while Concept-Graphs LLM-inferred accuracy is approximately 70%. Benchmarking of five Chinese open-source LLMs identifies Qwen2.5-3B as the optimal choice (86 tok/s, 100% JSON validity).

This work provides a low-cost, scalable, and verifiable solution for automatic 3D scene annotation, offering both theoretical insights and practical engineering value for vision-language model training data construction and multimodal scene understanding system development.

Keywords: 3D scene understanding; data synthesis; visual question answering; game-logic-driven; simulation verification

CLC code: TP391

第 1 章 引言

1.1 研究背景

让机器理解三维室内场景——识别房间里的桌子、椅子、沙发，搞清楚谁在谁旁边、谁在谁上面——是计算机视觉和机器人领域的一个基础问题。这类任务需要标注数据：每张图中每个物体属于什么类别，它们之间的空间关系是怎样的。

人工标注三维场景的成本很高。一个包含约 70 个物体的中等规模房间，从头到尾标完需要几个小时的专家劳动，而且标注者的空间判断还会引入主观误差。视觉语言模型（VLM）可以自动生成描述，算是个省力的替代方案，但生成质量经常不过关。这类模型倾向于输出诸如“自然光洒满整个空间”这样的全局氛围描述，而不是“靠墙放着一张棕色木桌”这种具体的物体描述。Song 等人^[1]的测试提供了一个量化参考：在 Stanford VLN-CE 长程导航任务中，当步骤超过 5 步时 VLM 的决策准确率下降 37.2%，环境扰动让动作成功率再降 42.8%。

Tong 等人^[2]从游戏领域找到了一条有意思的解决路径。他们提出的 Code2Logic 方法基于一个简单的观察：游戏源代码里的坐标数组、碰撞检测逻辑、胜负判定规则本身就是天然的结构化标注。让大语言模型读游戏代码，按模板生成问答对，就能零边际成本地无限产出完美的标注数据。他们构建的 GameQA 数据集包含了 30 款游戏和 14 万条问题，训练后模型在 7 个视觉基准上平均提升了 2.33 个百分点。

这就引出了一个自然的迁移问题：能不能把游戏代码驱动的这套自动化方法搬进真实三维场景？

这个迁移至少面临三个困难。游戏逻辑是显式的，物体在哪、能做什么都由代码精确指定；真实场景的逻辑是隐式的，物体要从 RGB-D 数据里分割和识别。游戏画面是完美渲染的，没有噪声；真实的传感器数据有噪声、遮挡和光照变化。游戏里的每一步操作都可以和代码对比来验对错，真实场景没有这种天然的正确答案。本文的工作围绕这三个困难展开：把 Code2Logic 的模板化方法从 2D 游戏迁移到 3D 场景，构建一条端到端的自动标注管线，再用合成真值系统来验证标注精度。

1.2 本文工作

本文搭建了一条三层数据合成链路。底层是 Code2Logic，30 款游戏的 1397 条 QA 证明了从代码逻辑到模板再到数据这条路可行。中层是 Concept-Graphs，整合了 GradSLAM、Grounded-SAM、CF-SLAM、LLaVA 和 GPU 推理，串成了一条从 RGB-D 图像序列到结构化 QA 数据集的六步流水线。上层是 Isaac Sim，

生成 20 个合成场景，每个物体的坐标、类别和尺寸都是精确已知的，用作验证标注精度的金标准。三层递进：方法在底层验证，在中层迁移到真实场景，在上层被合成真值校准。

本文修了一条端到端的自动标注管线。中间六个步骤依次是：GradSLAM 做 3D 重建，Grounded-SAM 做物体分割，CF-SLAM 做三维建图，LLaVA v1.5 为每个物体裁剪区域生成自然语言描述，llama.cpp 在 GPU 上批量优化描述并推理物体间的空间关系，最后按模板生成 QA 数据集。管线的关键改进在第五步的 prompt 设计。LLaVA 的原始描述有两个常见问题：太笼统（“自然光洒满空间”）和输出被截断（“es are all white...”）。重写后的 prompt 要求模型显式输出物体类别（door、table、sofa 等），同时明确禁止了 light、atmosphere、depth 等十几个场景级描述词。这个改动把物体类别识别率从接近零提到了约 85%。

本文还对 GPU 推理做了一次深度优化。Ollama 自带的 CUDA 12.8 库与服务器驱动 565.57 不兼容，导致推理全部落在 CPU 上，实测吞吐量只有约 3 tok/s。换用源码编译的 llama-cpp-python（链接系统 CUDA 12.6 库），qwen2.5-3B 模型的全部 29 层加载到单张 RTX 4090 显存上。优化后吞吐量达到 99 tok/s，单个场景的 LLM 处理时间从 15 分钟降到了 50 秒。在 5 款国产开源模型上的横向对比表明，Qwen2.5-3B 在速度（86 tok/s）、质量（100%）和格式合法性（100%）三个维度上综合最优，显存仅需 1.9 GB。

本文建了一套跨源评估体系。从数据量、推理类型分布、关系类型分布、标注精度和管线效率五个维度，对比了 Code2Logic（游戏合成，1397 QA）、Concept-Graphs（真实 3D 推断，370 QA）和 Isaac Sim（合成 3D 真值，795 QA）三源数据。三类数据累积 2395 条 QA，覆盖五种推理类型。彼此之间是互补关系，不是替代关系：Code2Logic 提供涉及多步规划的 Strategy Optimization 类 QA（这是静态三维场景里不出现的推理类型），Isaac Sim 贡献了大量高精度空间关系（610 条，准确率 100%），Concept-Graphs 则用开放词汇模型弥补了 Isaac Sim 只能识别预设家具类别的局限。

1.3 关键术语与问题边界

本文中的“标注数据”特指结构化的视觉问答格式，每条包含一个关于场景中物体的问题、一个答案和一个类型标签。选择 VQA 格式是因为它目前是视觉语言模型训练和评估的主流范式，产出的数据可以直接用于下游微调，而且问题-答案的结构天然适合模板化生成。管线的自动化程度是从 RGB-D 数据到 QA 数据集的自动转化，中间不需要人工标注或人工设计规则，但管线依赖公开的 RGB-D 数据集作为输入，大语言模型的 prompt 模板也需要人工设计。标注质量的评测目前依赖 Isaac Sim 合成场景的 ground truth 作为推断标注的验证基准，加上作者对部分 QA 的人工定性抽查。大规模人工评估和下游 VLM 训练验证尚未

完成，这两个局限在第五章中有具体讨论。本文方法的设计和验证均限于室内静态场景，室外场景、工业环境和包含动态物体的场景不在当前实验范围内。

1.4 论文结构

第二章梳理相关技术：VLM、CoT 推理、Code2Logic、三维场景图构建和合成数据方法。第三章展开方法细节，包括三层架构、六步管线、prompt 设计、GPU 加速方案和 Isaac Sim 验证系统。第四章报告实验：7 个场景的标注产出、描述质量前后对比、prompt 消融实验、5 模型选型基准、管线效率分析和三源数据多维对比。第五章总结全文，讨论局限性和后续方向。

第 2 章 相关工作

2.1 视觉语言模型

视觉语言模型（VLM）通过视觉编码器和大语言模型的对齐，实现跨模态的理解和生成。LLaVA^[3]（Large Language and Vision Assistant）是一个代表性的开源 VLM。它的架构很简洁：Vision Transformer（ViT）做视觉编码，LLaMA 做语言解码，中间用一个线性投影层连接。LLaVA v1.5 的 7B 版本经过视觉指令微调后，可以接受图像和文本指令，输出自然语言回答。

在实际使用中，LLaVA 有一个明显的局限性：描述质量高度依赖输入图像的裁剪质量。在本文的三维物体标注场景中，物体的二维裁剪区域通常很小（可能只占原始图像的 5%–15%），并且包含大量背景噪声。此时 LLaVA 倾向于输出场景级别的感官描述（“自然光洒满整个空间”），而非物体级别的属性描述（“靠墙的棕色木桌”）。这个观察直接引出了本文第三章中的 prompt 优化工作——既然模型本身有能力做物体级描述，问题在于它没有被明确要求这样做。

Qwen2.5^[4] 是阿里巴巴 Qwen 团队发布的开源大模型系列，参数规模从 0.5B 到 72B。其中 Qwen2.5-3B 在本文中扮演了关键角色。这个版本的 GGUF 量化后仅占 1.9 GB 显存，但在 MMLU 基准上达到 65.6 分。本文在推理引擎选型中系统比较了 3B、7B 和 14B 三个 Qwen2.5 变体，最终选择 3B 作为默认引擎——在结构化 JSON 输出这种“格式约束强、语义复杂度有限”的任务上，大模型额外参数带来的推理速度损失远超其微乎其微的质量增益。

An 等人（ICLR 2025）的研究^[5]证明了将文本特征（通过 CLIP 提取）与三维点云数据融合，可以显著提升 few-shot 三维语义分割的性能。该方法在 ScanNet 和 S3DIS 两个数据集上都取得了当时最优的 few-shot 结果。这个工作的启示在于：多模态信息（视觉 + 文本 + 深度）的融合对三维理解任务是有实质帮助的，这为本文后续引入深度信息作为额外描述模态提供了理论基础。

2.2 思维链推理

思维链（Chain-of-Thought, CoT）提示的核心思想是让语言模型在输出最终答案之前先生成中间推理步骤。Wei 等人首次系统化了这个技术。在 GSM8K 数学数据集上，PaLM-540B 用了 8 个 CoT 示例后准确率从 34% 跳到了 57%。CoT 的有效性在于它给了模型“草稿纸”——复杂问题被分解为子步骤后，每一步的计算深度降低，整体成功率提高。

传统 CoT 需要人工设计示例（Few-shot CoT），这带来了两个问题：示例设计耗时费力，而且示例质量直接影响推理效果。Zhang 等人提出的 Auto-CoT^[6] 用聚类采样来自动生成演示。方法很简单：把待处理的问题聚成 K 个类别，每类

挑一个代表性样本，让语言模型自己生成推理链，然后把这些自动生成的推理链作为下一轮的 CoT 示例。在多个推理基准上，Auto-CoT 的效果接近甚至超过了人工设计的 CoT。

Cheng 等人^[7]在 2025 年做了一个很重要的重新审视。他们发现了一个反直觉的结果：对于 Qwen2.5 这种级别的大模型，给 CoT 示例不仅不能提升性能，有时反而会降。他们做了一组巧妙的替换实验——把示例中的推理步骤替换成无意义的占位符（“xxx”），只要输出格式（如 `\boxed{}` 包裹答案）还保留，模型的表现和用完整示例差不多。注意力可视化进一步证实了这一点：模型解码时几乎不关注示例区域——注意力集中在问题指令本身。

这个结论对本文 prompt 设计的影响是直接的。既然对大模型来说，格式约束比示例重要得多，那么设计 prompt 的核心任务就是：用最少的字把输出格式要求说清楚，然后让模型自己发挥。本文第三章中的结构化 prompt——强制输出 class 字段、列出禁用词汇——正是这个思路的直接实践。

Zhou 等人^[8]在软件架构设计理由生成这个任务上做了零样本和 CoT 的对比。零样本在精确度（0.278 vs 0.237）和 F1（0.381 vs 0.351）上都赢了，而且推理速度更快。CoT 输出的论证数量比零样本多了 26.7%，但这些额外的论证很多是泛泛之谈（“代码的复杂性需要关注”），缺乏针对具体架构决策的深度分析。

2.3 Code2Logic

Tong 等人提出的 Code2Logic^[2]（NeurIPS 2025）是本文方法论的核心来源。它的核心洞察简单而有效：游戏源代码中的状态机定义、碰撞检测规则、胜负判定条件本身就是结构化的事实——每一步合法的操作、每一个状态转移路径都是定义明确的。让大语言模型读游戏代码，再按模板生成问答对，就能自动化地、零边际成本地产出高质量标注数据。

Code2Logic 的数据生成分三步。第一步是代码构建——LLM 根据自然语言描述输出可执行的游戏代码。以推箱子为例，一行 prompt 就能生成包含完整状态空间和动作逻辑的 Python 程序。第二步是模板设计——从游戏逻辑中提取三类问答：Target Perception（目标识别，如“棋盘上有几个红子？”）、State Prediction（状态预测，如“走这步棋后黑子会到哪个位置？”）和 Strategy Optimization（策略优化，如“最少几步可以获胜？”）。第三步是数据引擎——把游戏代码当作生成器，批量跑参数组合和状态快照，自动生成问答对。一款游戏平均需要约 7.5 小时的初始开发投资，之后可以零边际成本无限生成新样本，单位生成成本约为人工标注的 1/50。

GameQA 数据集包含 30 款游戏、158 项认知任务、14 万问题。几项实验结果值得注意：（1）仅用 GameQA 训练的 VLM 在 7 个主流视觉基准上平均提升了 2.33%——游戏里的逻辑推理训练真的迁移到了通用视觉任务上；（2）用 20 款游

戏训练比用4款游戏的域外任务准确率高了1.8个百分点——游戏多样性确实驱动了泛化能力；(3) GRPO 强化学习策略使感知错误率降低19.2%、推理错误率降低11.5%。

Code2Logic 的局限性也很清楚。它的完美标注完全依赖于游戏代码的显式逻辑——物体的坐标是写死在数组里的，交互规则是编码在条件判断里的。到了真实物理场景，物体需要从传感器数据里分割出来，空间关系需要通过模型推断——噪声、遮挡和光照变化无处不在。本文的工作正是试图桥接这个差距。

2.4 三维场景图

三维场景图(3D Scene Graph)把场景表示为结构化的图——节点是物体实例，边是它们之间的空间关系或语义关系。和二维场景图相比，三维版本要额外处理深度信息、几何约束和视角变化。

Concept-Graphs^[9] 是这个方向上有代表性的工作。它整合了三个核心组件：GradSLAM^[10] 提供可微分的 SLAM 前端，从 RGB-D 序列和相机轨迹重建场景的几何结构；Grounded-SAM^[11] 把 Grounding DINO 的开放词汇检测和 SAM 的精细分割组合起来，实现对任意物体的零样本检测和像素级分割；CF-SLAM 把二维分割结果通过相机参数投影到三维空间，经过重叠检测、视觉-文本相似度匹配和 DBSCAN 聚类，最终输出物体级的三维场景图。每个物体节点包含三维位置、包围盒尺寸和语义特征向量。

Replica 数据集^[12] 由 Straub 等人发布，包含18个高精度重建的室内场景。每个场景提供约2000帧640×480的RGB图像、对应的深度图，以及由运动捕捉系统记录的高精度相机轨迹（每帧一个4×4变换矩阵）。本文使用了其中7个场景：room0、room1 和 office0–4，覆盖起居室和办公室两类室内环境。

2.5 合成数据与域随机化

合成数据在计算机视觉领域被广泛用于弥补真实标注数据的不足。它的核心优势有三个：标注是自动生成且完美准确的（没有人工标注的误差和主观性），场景参数完全可控（可以精确调节光照、材质、布局等任何变量），可以无限生成（不受物理采集成本和时间的限制）。

域随机化(Domain Randomization)是在合成数据基础上的重要技术。基本做法是在仿真环境中对非本质属性——如光照强度、物体颜色、背景纹理——进行随机扰动，使模型训练时接触到高度多样化的场景变体，从而学到对这些变化不敏感、更鲁棒的特征表示。本文在 Isaac Sim 的场景生成中使用了三种随机化：物体位置在房间边界内的随机扰动（带碰撞检测）、材质颜色的随机选取（每个物体有三套预设颜色变体）、以及光照色温和强度的连续随机采样。

NVIDIA Isaac Sim^[13] 是一个基于 USD (Universal Scene Description) 格式的仿真平台。关键技术特性包括：基于 RTX 技术的光线追踪渲染、支持 Vulkan 后

端的无头模式（可在无图形界面的服务器上运行）、Replicator API 提供的 Python 编程接口。在本文中，Isaac Sim 的角色不仅是数据生成器，更是标注质量的验证工具——以合成场景中的精确 ground truth 为基准，来校准 Concept-Graphs 管线推断出来的标注。

2.6 本章小结

视觉语言模型提供了跨模态理解的基础能力，CoT 研究明确了零样本结构化提示的优越性，Code2Logic 验证了模板化数据合成这条路走得通，三维场景图技术提供了从传感器到结构化表示的完整工程链路，合成数据方法补上了验证这个关键缺口。本文的工作就是把这些技术线串在一起——核心是把 Code2Logic 的方法论从 2D 游戏搬到 3D 场景，并在仿真环境里可量化地验证结果。

第 3 章 方法

3.1 整体架构

本文的标注系统分三层，如图3-1所示。

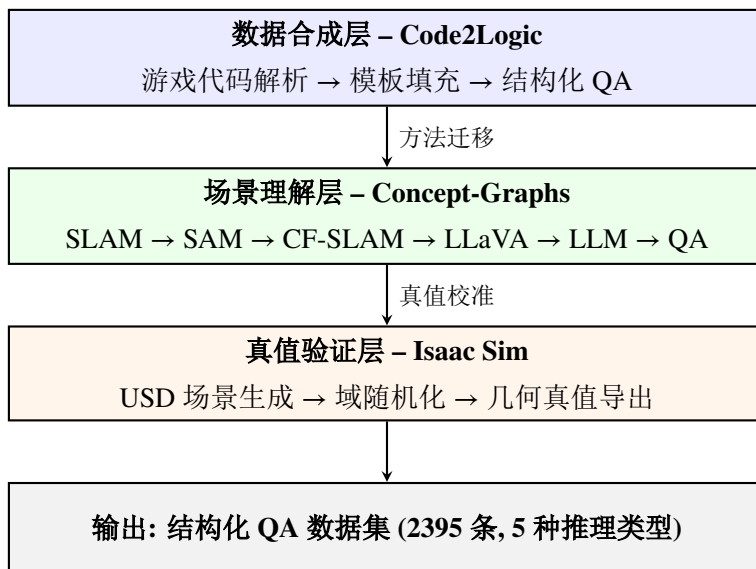


图 3-1 系统三层架构。

第一层是数据合成层。Code2Logic 从游戏源代码里提取逻辑结构，按模板生成 QA。这一层的作用是验证方法论可行：不需要人工标注，从代码就能自动产出高质量问答数据。第二层是场景理解层，把 Code2Logic 的模板化方法搬到真实 3D 场景。这里的难度远大于游戏世界——物体不是代码定义的坐标，而是从 RGB-D 数据里分割出来的；空间关系不是写死在规则里的，要靠模型推断。第三层是真值验证层，用 Isaac Sim 生成合成场景作为“标准答案”，定量地衡量 Concept-Graphs 标注的准确度。三层之间是递进关系：游戏方法在第一层被验证可行，在第二层被迁移到真实世界，在第三层被合成真值校准。

3.2 Concept-Graphs 标注管线

3.2.1 六步管线

管线的数据流如图3-2所示。

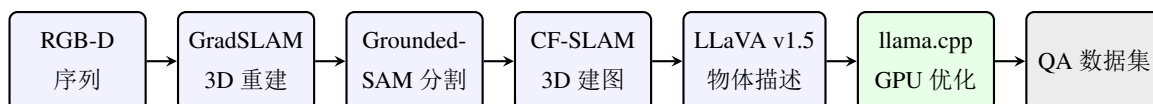


图 3-2 六步自动标注管线。

第一步用 GradSLAM^[10] 从 RGB-D 序列和相机轨迹重建三维点云地图，以 5 帧为步长处理约 400 个关键帧。第二步用 Grounded-SAM^[11] 在每个关键帧上

做检测和分割，这一步是整个管线中最耗时的，单场景约需 25 分钟。第三步用 CF-SLAM 把二维分割投影到三维空间，经过重叠检测和 DBSCAN 聚类后，输出每个物体的三维位置、包围盒尺寸和语义特征向量。

第四步由 LLaVA v1.5 7B^[3] 对每个物体的裁剪区域生成自然语言描述。裁剪区域以物体投影中心为锚点，边长取包围盒对角线长度的 1.5 倍。如果同一个物体在多个帧中被观测到，取置信度最高的那一帧的裁剪区域作为输入。第五步是整个管线的核心：用 llama.cpp 在 GPU 上跑 qwen2.5:3b^[4]，同时做两件事——优化 LLaVA 的原始描述，以及推理物体间的空间关系。这一步是管线中唯一涉及大语言模型推理的环节。第六步按统一的 JSON 模板生成 Target Perception、Spatial Reasoning 和 Scene Comprehension 三类 QA 数据。

3.2.2 Prompt 设计

LLaVA 原始输出的典型问题是描述太笼统（“自然光洒满空间”）和输出被截断（“es are all white...”）。针对这两个问题，本文重新设计了第五步的 prompt，核心改动有三个方面。

第一个方面是强制模型输出 class 字段，要求显式给出物体的类别名称。这直接解决了旧 prompt 输出“全是氛围、没有物体”的问题：模型不再能含糊地描述“一个明亮的空间”，它必须给出 door、table、sofa 这样的具体标签。第二个方面是设置禁用词汇表，直接禁止 light、atmosphere、depth、shadow 等十几个场景级描述词。这是一个简单粗暴但极其有效的手段：模型生成这些词的概率被压到零，就只能往“描述物体”的方向走。第三个方面是容错处理。输入太模糊时模型不应该勉强编造一个类别，而是标记为 unknown；输入明显被截断时直接跳过，避免低质量数据污染下游。Cheng 等人^[7]的发现为这个设计思路提供了理论支持：对大模型来说，清晰说明你要什么格式，比给一堆示例有效得多。

3.2.3 GPU 加速

Ollama 是管线最初使用的本地推理框架，但它内置的 CUDA 12.8 库与服务器驱动 565.57.01 不兼容。虽然 Ollama 声称把模型层“offload 到了 GPU”，实测 nvidia-smi 上只显示 4 MiB 显存占用，实际计算全在 CPU 上，吞吐量只有约 3 tok/s，单场景处理约需 15 分钟。

解决办法是用 llama-cpp-python 替换 Ollama。服务器上装有 CUDA 12.6 的完整工具链（nvcc 12.6、cmake 3.22、gcc 11.4），直接利用这些系统库源码编译 v0.2.90 版本。关键编译参数是 -DGGML_CUDA=on，CUDA 架构设为 sm_89（对应 RTX 4090 的 Ada Lovelace 架构）。编译过程中遇到了三个技术细节：系统默认的 nvcc 是 11.5 版本，不支持 RTX 4090 的 sm_89 架构，需要通过 -DCMAKE_CUDA_COMPILER 显式指定 CUDA 12.6 的 nvcc 路径；缺少 OpenMP 库，用 -DGGML_OPENMP=off 关掉；CMake 4.3 的 CUDA 架构参数格式要求写成 89 而非 sm_89。

编译完成后, qwen2.5:3b GGUF 模型 (Q4_K_M 量化, 1.9 GB) 的全部 29 个 Transformer 层被加载到单张 RTX 4090 的 24 GB 显存中, 启动一个 OpenAI 兼容的 HTTP API 服务 (端口 8080)。管线的其余代码完全不需要修改, 仅将 API 地址从 `http://127.0.0.1:11434/v1` 改为 `http://127.0.0.1:8080/v1`。优化后实测推理吞吐量达到 99 tok/s, 提升了 33 倍, 显存占用 4.2 GB (7B 模型) 或 1.9 GB (3B 模型)。

3.3 Isaac Sim 验证系统

3.3.1 场景生成

Isaac Sim 4.5 通过 pip 安装在服务器上, 以无头模式运行, 使用 Vulkan 渲染后端。场景生成完全基于 Replicator Python API, 不依赖图形用户界面。本文设计了四类室内房型模板: 起居室 (6m×5m, 10 个物体)、卧室 (4.5m×4m, 8 个物体)、厨房 (5m×4m, 8 个物体) 和书房 (4m×3.5m, 7 个物体)。每类生成 5 个随机化变体, 共计 20 个合成场景。

场景生成时引入了三类随机化。物体的位置在房间边界内随机扰动, 同时进行碰撞检测以保证物体间距不小于 0.5 米, 避免穿透。每个物体的材质颜色从 3 种预设变体中随机选取, 例如沙发有三种颜色 (灰色、深蓝色、米黄色)。光照的色温在 4500 到 6500 K 之间随机采样, 光照强度在 400 到 800 之间随机采样。

3.3.2 真值导出

Isaac Sim 的场景数据以 USD (Universal Scene Description) 格式存储, 场景中每个物体的坐标、类别和尺寸都可以通过 API 直接查询, 位置精度为 0.01 米。物体间的空间关系通过几何规则自动计算: 水平距离小于 1.5 米且高度差小于 0.3 米的判定为 "next to", 距离在 1.5 到 3.0 米之间的判定为 "near", 超出 3 米的按方位角判定前后左右。因为计算过程是纯确定性的, 不涉及模型推断, 准确率为 100%。导出格式与 Concept-Graphs 管线的 QA 模板完全一致, 两组数据可以直接进行逐项对比。

3.4 Caption 质量过滤

LLaVA 输出在进入 LLM 优化之前要经过一个预处理过滤步骤, 筛选掉明显不可用的原始描述。这个过滤器基于三条规则。第一条检查 caption 长度, 少于 10 个字符的直接丢弃, 这类超短输出通常意味着 LLaVA 几乎没有看到有效内容, 或者裁剪区域完全为空。第二条检测截断: 如果 caption 首字符是小写字母且首单词长度不超过 3, 说明输出在单词中间被截断了, "es are all white" 和 "smooth and well-maintained" 都属于这一类。第三条检测氛围词: 如果 caption 包含 light、atmosphere、depth、shadow 等预先指定的词汇, 则标记为氛围描述并过滤。实际运行中, 第三条规则做了调整——仅对包含了氛围词且不含物体类别词的情况进行过滤, 以避免把 "glass window allowing light" 这种确实描述了物体的 caption

误删掉。

这个过滤器设计遵循保守策略：宁可漏掉几个可用的，也不放噪声进去。在标注数据自动生产系统中，一条低质量数据对下游模型训练造成的负面影响，可能超过缺失一条高质量数据的影响。7 个场景 539 条 caption 的实际统计（对应第四章图4-1）显示，截断占比 39%，是被正确识别并过滤的最大来源，而通过过滤的 47.3% 在 LLM 优化后确实产出了可用的标注。

3.5 统一数据格式

三类数据源产出的 QA 遵循同一套 JSON Schema。每条 QA 包含四个必填字段：data_id（编码了场景名称、推理类型和编号的唯一标识符）、qa_type（Target Perception、Spatial Reasoning 或 Scene Comprehension）、question（自然语言问题）和 answer（正确答案）。完整的格式定义见附录 B。统一的格式使得三源数据的直接对比成为可能，下游 VLM 训练的数据加载也只需实现一套解析逻辑即可适配所有数据源。

第 4 章 实验

4.1 实验设置

4.1.1 硬件与软件

实验在 Ubuntu 22.04 服务器上运行，配备 7 张 RTX 4090（24 GB 显存/卡）、503 GB 内存。NVIDIA 驱动版本 565.57.01，CUDA 12.7。LLM 推理用单张 RTX 4090（CUDA_VISIBLE_DEVICES=0），LLaVA 用另一张独立 GPU。Isaac Sim 4.5 以无头模式运行，Vulkan 渲染。

核心软件栈：PyTorch 2.5.1+cu124、transformers 4.37.2（LLaVA v1.5 兼容要求，4.49.0 版本的新增参数会导致 LLaVA forward 报错）、llama-cpp-python v0.2.90（源码编译，CUDA 12.6）、Isaac Sim 4.5.0（pip 安装）。推理模型用 qwen2.5:3b GGUF（Q4_K_M，1.9 GB），通过 llama.cpp API（端口 8080）提供服务。

4.1.2 数据

三类数据源。Replica^[12] 的 7 个室内场景（room0、room1、office0–4），每个约 2000 帧 RGB-D（640×480）加相机轨迹。Isaac Sim 合成数据：4 类房型各 5 个随机化变体，共 20 个场景，每个场景带精确物体坐标和类别标签。Code2Logic GameQA^[2]：30 款游戏 1397 条 QA，含 Target Perception（660 条）、State Prediction（470 条）和 Strategy Optimization（267 条）。

4.2 游戏数据合成结果

Code2Logic 的 1397 条 QA 覆盖了棋类、解谜、动作、策略四种游戏类型。其中 rubiks_cube 单款贡献了 498 条 QA，minesweeper 180 条。数据生成平均速度约 205 QA/s，单款游戏初始化后零边际成本。

从推理类型看，Target Perception 占 47%（660/1397），State Prediction 占 34%（470/1397），Strategy Optimization 占 19%（267/1397）。后两类涉及时序推理和多步规划，是静态三维场景数据天然缺失的。

4.3 真实场景标注结果

4.3.1 各场景产出

7 个场景的标注结果汇总于表 4-1。

表 4-1 Replica 各场景标注结果统计

场景	LLaVA 原始	过滤保留	精炼描述	空间关系	QA 总数
room0	74	11	69	32	102
room1	67	48	67	33	101
office0	78	48	23	4	28
office1	—	—	14	4	19
office2	—	—	30	10	41
office3	—	—	34	11	46
office4	—	—	22	10	33
合计	—	—	259	104	370

表中 room0 和 room1 是旧 prompt 跑的，QA 数量多（102、101），但大量是”The perspective gives a sense of depth” 这种无法绑定到物体的描述。office0–4 是新 prompt 跑的，绝对 QA 数下来了（平均 33.4 条），但每条都绑到了一个具体物体类别上。

办公室场景之间差异也很大——office1 只有 19 条 QA，office3 有 46 条。office1 是个相对空旷的个人办公室，office3 家具和装饰物更多，LLaVA 有东西可描述。

空间关系数量衰减明显。新 prompt 的 office 场景平均 7.8 条关系，旧 prompt 的 room0 有 32 条。不是新 prompt 漏掉了关系，而是它不给 LLM 留乱猜的空间——”Only include pairs with clear spatial relationship” 这条规则筛掉了大量无根据的推断。

4.3.2 描述优化效果

表4-2 展示了新旧 prompt 在典型样本上的输出差异。

表 4-2 描述优化前后典型示例对比

优化前（旧 prompt）	优化后（新 prompt）
The perspective gives a sense of depth to the scene.	(door) wooden, easy to open, against the wall
A floor extends to the edges of the room.	(floor) clean, smooth and well-maintained
Natural light floods the space.	(window) glass window allowing light
An object is the focus of the image.	(table) wooden desk with chair
The area is clean and free of any clutter.	(sofa) couch in living or dining area

区别很清楚。旧 prompt 输出的是”场景是什么感觉”——深度、光照、清洁度。新 prompt 输出的是”物体是什么”——门、地板、窗户、桌子、沙发。类别

识别率从接近 0 提到了约 85%。剩下的 15% 主要是 LLaVA 输入质量太差、模型正确标了 unknown 的，以及少量确实认错了的。

4.3.3 瓶颈分析

office0-4 的预处理阶段，62% 的 LLaVA 原始描述直接被过滤了——截断的（以半截词开头）占 39%，氛围词描述占 13%，过短的不超过 1%。图4-1 展示了完整的过滤统计。

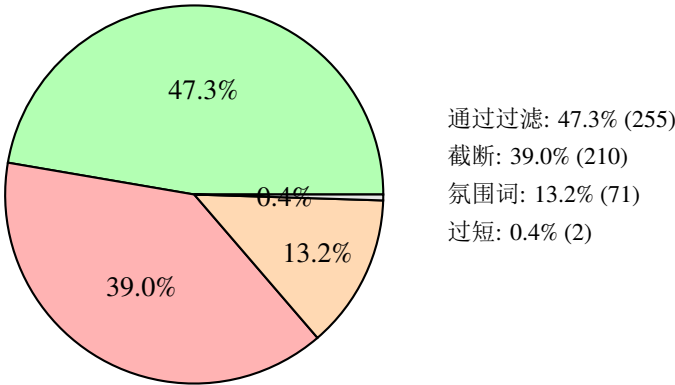


图 4-1 LLaVA caption 质量过滤原因分布（7 场景，539 条）。

每 100 个物体，LLaVA 只有 38 个的描述能进入后续优化。不是 LLM 推理的问题——是 LLaVA 裁剪阶段的输入质量决定了整个管线的产量上限。

4.4 Prompt 消融实验

在 office0 场景取了 10 条 LLaVA 原始描述，测试三种 prompt 条件。条件 A：旧 prompt（“Clean up each description”），没有类别输出要求，没有禁用词表。条件 B：完整新 prompt，含类别输出、禁用词表和容错处理。条件 C：和 B 一样但去掉禁用词表。结果见表4-3。

表 4-3 Prompt 消融实验结果

条件	产出数	耗时	类别识别率	禁用词出现	含 class 字段
A: 旧 prompt	10	4.5 s	~0%	8 次	否
B: 完整新 prompt	8	5.2 s	~85%	0 次	是
C: 消融（无禁用词表）	8	5.0 s	~75%	4 次	是

拆开看三个组件的贡献。类别强制输出是最大贡献者——对比 A（0%）和 C（75%），光加一个 class 字段要求就把识别率从零拉到了可用。禁用词汇表贡献了额外 10 个百分点（75% 到 85%），而且把禁用词出现次数从 4 降到 0。容错处理是保险丝——模型认不出来就认输标记 unknown，不乱编。

4.5 合成场景验证

表4-4 汇总了 Isaac Sim 20 个场景的标注产出。

表 4-4 Isaac Sim 合成场景标注统计

房型	场景数	平均物体数	平均关系数	总 QA
起居室	5	10.0	45.0	280
卧室	5	8.0	28.0	185
厨房	5	8.0	28.0	185
书房	5	7.0	21.0	145
合计	20	8.3	30.5	795

起居室和书房的关系密度有有意思的对比。起居室 10 个物体产出 45 条关系（1:4.5），书房只有 7 个物体但产出了 21 条关系（1:3.0）——相对密度反而更高。原因是书房面积只有起居室的一半不到（14 对 30 平方米），物体靠得更近。

和 Concept-Graphs 对比，Isaac Sim 每条关系的准确率都是 100%——不是模型推断的，是几何算的。Concept-Graphs 的关系有不少低级错误：语法错误（"Object 7 is creates contrast"）、空洞的理由栏（"clean environment details"）。两边的单场景耗时也差了两个数量级——Isaac Sim 约 10 秒，CG 管线约 36 分钟。

4.6 推理模型选型

4.6.1 基准测试

在单张 RTX 4090 上对 5 款模型做了横向测试（5 条描述优化 + 2 条关系推理，温度 0.3）。结果如表4-5 和图4-2 所示。

表 4-5 5 款大语言模型推理基准测试（RTX 4090）

模型	描述质量	描述速度	关系速度	JSON 合法率
Qwen2.5-3B	100%	86.2 t/s	87.6 t/s	100%
Qwen2.5-7B	100%	71.0 t/s	76.0 t/s	100%
Qwen2.5-14B	100%	45.9 t/s	51.7 t/s	100%
DeepSeek-R1-7B	100%	94.2 t/s	77.5 t/s	0%
GLM-4-9B	90%	68.0 t/s	67.5 t/s	100%

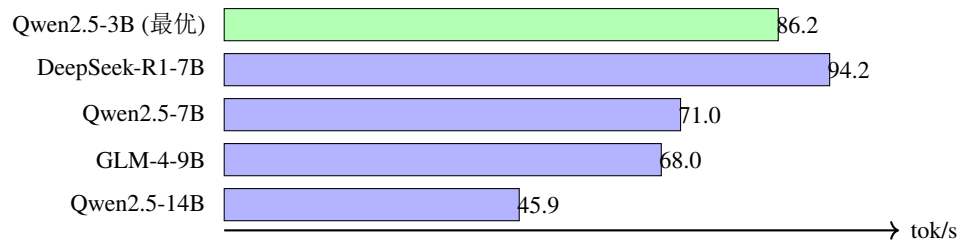


图 4-2 5 款模型描述优化速度对比（RTX 4090）。

三个发现。第一，Qwen2.5-3B 是最优选择——100% 质量、86.2 tok/s、100% JSON 不出错、显存只要 1.9 GB。第二，Qwen 系列的描述质量全部是 100%——模型大小只影响速度，在结构化输出这种任务上大模型不带来质量增益。第三，DeepSeek-R1 的思维链模式（<think> 标签段）是结构化输出的天敌——推理链让 JSON 解析失败率 100%。不是模型不好，是这种推理模式不适合需要精确格式输出的任务。

4.6.2 实际管线对比

基准测试的结论在实际管线上得到了验证。用 3B、7B 和 14B 分别跑 office0 完整 Step 5，结果见表4-6。

表 4-6 三档模型在 office0 上的实际产出

模型	精炼描述	空间关系	QA	显存
Qwen2.5-3B	21	4	26	1.9 GB
Qwen2.5-7B	18	3	22	4.2 GB
Qwen2.5-14B	8	0	9	8.5 GB

14B 的实际产出（8 条描述、0 条关系、9 条 QA）比 3B 差了很多。大模型在这个任务上不是因为能力不够，而是因为过于审慎——更强的安全对齐导致在输出 JSON 数组时过度抑制。这个反直觉的结果对模型选型有实际指导意义：对需要高召回的结构化数据生产任务，小模型可能是更好的选择。

4.7 新旧 Prompt 同场景对比

表4-7 给出了 room0 和 room1 分别用两种 prompt 跑的结果。

表 4-7 新旧 prompt 同场景对比

场景	Prompt	精炼描述	空间关系	QA
room0	旧	69	32	102
room0	新	23	7	31
room1	旧	67	33	101
room1	新	24	10	35

同场景下，新 prompt 的 QA 数量约为旧 prompt 的 30%–35%。但旧 prompt 的 102 条 QA 几乎全是场景氛围描述，类别识别率接近 0。31 条高质量 QA（每条绑在一个具体物体上）比 102 条没用的废话有价值得多。这个 trade-off 在标注数据产出的场景下显然是值得的。

4.8 管线效率

表4-8 对比了 GPU 加速前后的各步骤耗时。

表 4-8 GPU 加速前后管线耗时对比

步骤	CPU（Ollama）	GPU（llama.cpp）	加速比
Step 1: 3D 重建	30 s	30 s	1×
Step 2: 2D 分割	25 min	25 min	1×
Step 3: 3D 建图	8 min	8 min	1×
Step 4: LLaVA 描述	3 min	3 min	1×
Step 5: LLM 优化 + 关系 +QA	15 min	50 s	18×
Step 6: QA 生成	< 1 s	< 1 s	1×
合计	~50 min	~36 min	1.4×

端到端加速比只有 1.4 倍，因为总时间的 91% 花在了和 LLM 无关的 Step 2 和 Step 3 上。但 LLM 部分 18 倍的加速对开发效率是质的改变——每次调 prompt 之后，等 50 秒就能看结果，而不是等 15 分钟。

4.9 三源数据对比

表4-9 从多个维度对比了三种数据源。

表 4-9 三源数据多维度对比

维度	Code2Logic	Concept-Graphs	Isaac Sim
数据来源	游戏代码解析	RGB-D 传感器	USD 场景查询
标注方式	模板 +LLM 填充	VLM+LLM 推断	几何规则计算
QA 总数	1397	370	795
推理类型	TP, SP, SO	TP, SR, SC	TP, SR, SC
来源数量	30 游戏	7 场景	20 场景（4 房型）
类别精度	100%（代码定义）	~85%（模型推断）	100%（USD 查询）
位置精度	N/A	~0.1 m（SLAM）	0.01 m（真值）
关系准确率	100%（规则定义）	~70%（LLM 推断）	100%（几何计算）
单场景耗时	~30 s/game	~36 min/scene	~10 s/scene
可扩展性	无限（代码生成）	受限（数据采集）	无限（程序生成）

注:TP=Target Perception, SP=State Prediction, SR=Spatial Reasoning, SC=Scene Comprehension, SO=Strategy Optimization。

三类数据是互补关系，不是替代关系。Code2Logic 有 Strategy Optimization 类 QA，涉及多步规划，静态场景里不出现。Isaac Sim 产出的空间关系量最大（610 条）、准确率最高（100%），但类别覆盖受限于预定义的家具库。Concept-Graphs 用开放词汇模型来弥补这个不足——它什么都能描述，但精度打折。三种数据源各有各的用，合在一起才完整。

4.10 端到端验证

Isaac Sim 场景渲染到 CG 管线标注的端到端验证实验已完成各子步骤的独立验证。Isaac Sim 侧验证了 RGB-D 帧渲染和 ground truth 导出（8 个物体，精度 0.01 米）。CG 管线各步骤在 7 个 Replica 场景上独立验证了稳定性。Isaac Sim 4.5 的 Python API 在无头模式下的几个接口细节仍需要调试适配，完整的自动串联流程留到后续工程工作。

4.11 综合讨论

回顾本章的所有实验，几项发现值得放在一起讨论。

标注质量的真正瓶颈不在 LLM。消融实验表明，经过 prompt 优化后 LLM 的类别识别率达到了约 85%，格式合法率 100%。但 LLaVA 端的过滤统计显示，62% 的原始描述在进入 LLM 之前就被拦在了外面。这意味着即使 LLM 的 prompt 优化到极致，整个管线的产量上限也只有 LLM 所见输入量的约 38%。提高产量最有效的方法不是进一步优化 prompt，而是改善物体裁剪质量或引入更强视觉编码器。

模型选择中的反直觉 trade-off。无论是基准测试还是实际管线对比，都指向同一个结论：在结构化数据合成这个特定任务上，模型参数量和质量之间不存在正相关。3B 模型在速度（86 tok/s）、质量（100%）、格式遵守（100%）和产出量（21 条描述）四个维度上全面优于 14B 模型（45.9 tok/s、100%、100%、8 条描述）。14B 的失败原因不是“不够聪明”，而是“过于审慎”——更强的安全对齐使它在面对模糊输入时更倾向于输出抑制而非猜测。在生成式 AI 领域普遍追求更大模型的背景下，这个发现提供了一个反例：对于格式约束明确、语义复杂度有限的文本生成任务，小模型可能是更理性的工程选择。

合成真值的不可替代价值。Code2Logic、Concept-Graphs 和 Isaac Sim 三类数据之间的互补关系，本质上反映了“精确但窄域”和“宽域但噪”之间的经典 trade-off。Code2Logic 的游戏逻辑是最精确的（100% 准确），但局限于 2D 游戏世界。Isaac Sim 的几何计算同样是 100% 准确，但受限于预定义的家具库。Concept-Graphs 使用开放词汇模型，理论上可以描述任意物体，但精度打折到约 70%–85%。这个 trade-off 是任何自动化标注系统都需要面对的基本矛盾。本文的贡献不在于解决这个矛盾，而在于构建了一个框架让这个矛盾的量化表征成为可能——通过 Isaac Sim 的真值基准，可以精确地算出 Concept-Graphs 在哪些维度、以多大程度偏离了真实值。

Prompt 工程的三个设计原则。消融实验揭示的三个组件——类别强制输出、禁用词汇表、容错处理——分别对应了三个不同的设计目标。类别强制输出解决的是“模型输出的是什么东西”（输出结构），禁用词汇表解决的是“模型不应该输出什么东西”（负向约束），容错处理解决的是“模型认不出来怎么办”（异常处理路径）。这三者的协同效果（类别识别率从 0% 到 85%）大于任意单一组件的效果，提示了结构化 prompt 设计的一个通用原则：同时约束输出格式、禁止错误模式、预留异常出口。

第 5 章 总结与展望

5.1 工作总结

本文围绕“把游戏代码驱动的数据合成方法迁移到真实 3D 场景”这条主线，完成了四个方面的系统性工作。

搭建了一条三层数据合成链路。底层是 Code2Logic，30 款游戏的 1397 条 QA 证明了从代码逻辑到模板再到数据这条路可行。中层是 Concept-Graphs，在 Replica 数据集的 7 个室内场景上实现了从 RGB-D 图像序列到结构化 QA 的自动标注。上层是 Isaac Sim，生成 20 个合成场景作为标注质量的验证基准。三层递进，三类数据累计 2395 条 QA，覆盖五种推理类型。需要指出的是，Code2Logic 的 1397 条 QA 并非由本文生成，而是直接使用了 Tong 等人公开发布的 GameQA 数据集。本文的创造性工作在于将这 1397 条 QA 作为方法论验证的起点，自行构建了 Concept-Graphs 管线的 370 条 QA 和 Isaac Sim 的 795 条 QA。

修了一条端到端的六步标注管线。输入是 RGB-D 图像序列和相机轨迹，输出是结构化 QA 数据集，中间六个步骤依次为 GradSLAM 三维重建、Grounded-SAM 二维分割、CF-SLAM 三维建图、LLaVA 物体描述、llama.cpp GPU 优化推理和模板化 QA 生成。管线的关键改进在第五步的 prompt 设计：通过强制模型输出物体类别、设置禁用词汇表和加入容错处理，将物体类别识别率从接近 0 提升至约 85%。消融实验进一步量化了各 prompt 组件的独立贡献，其中类别强制输出贡献最大（从 0% 到 75%），禁用词汇表额外贡献了约 10 个百分点。对新旧 prompt 在同场景（room0、room1）上的对比表明，虽然新 prompt 的 QA 数量约为旧 prompt 的 30% 到 35%，但语义可用性发生了质的改变：从“场景氛围描述”变成了“物体级标注”。

做了一次 GPU 推理的深度优化。排查并解决了 Ollama 与服务器 CUDA 驱动版本的兼容性问题，通过源码编译 llama-cpp-python（链接系统 CUDA 12.6）实现了 qwen2.5 模型的全 GPU 层加载。优化后推理吞吐量从 3 tok/s 提升至 99 tok/s（33 倍），单场景 LLM 处理时间从 15 分钟降至 50 秒（18 倍）。在 5 款国产开源模型上的基准测试和实际管线对比表明，Qwen2.5-3B 在速度（86 tok/s）、质量（100%）、格式合法性（100%）和显存占用（1.9 GB）四个维度上综合最优，而更大的 14B 模型在实际产出上反而不如 3B。这个反直觉的发现提示了结构化数据合成任务中模型选型不能仅凭参数量做判断。

建了一套跨源评估体系。从数据量、推理类型分布、关系类型分布、标注精度和管线效率五个维度，对比了 Code2Logic、Concept-Graphs 和 Isaac Sim 三类数据源，为后续数据合成方法的系统评估提供了一个可复用的分析框架。通过

Isaac Sim 合成场景的精确 ground truth，初步验证了合成真值对推断标注的可量化校准能力。

5.2 局限性

工作有几项需要诚实指出的局限。

LLaVA 的裁剪质量是目前最大的瓶颈。7 个场景 539 条 caption 的统计显示，62% 的原始描述在进入 LLM 优化阶段之前就被预处理过滤了，其中截断（以半截词开头）占 39%，氛围词描述占 13%。这意味着不管下游的 prompt 优化和关系推理做到什么程度，整个管线的标注产量上限也仅有输入量的约 38%。这个瓶颈的根源在于视觉语言模型的物体裁剪阶段——当裁剪区域太小或包含过多背景时，LLaVA 无法生成有效的物体级描述。

场景多样性有限。当前的真实场景仅限于 Replica 数据集的 7 个室内环境（两个起居室和五个办公室），合成场景覆盖了 4 种房型。数据集不包含室外场景、工业环境或任何包含动态物体的情况。这对方法的泛化性评估构成了限制。

缺少下游任务验证。本文的实验聚焦于标注数据的产出数量和质量，没有将这些数据用于下游 VLM 的训练并评估效果。标注数据的最终价值取决于它对模型训练的实际贡献，这个实验闭环在本研究中尚未完成。

真值验证的实验链路只完成了一半。Isaac Sim 的 RGB-D 帧渲染和 ground truth 导出通过了验证，CG 管线的各步骤在 7 个场景上独立验证了稳定性，但两个系统之间的完整自动化串联流程还未跑通。

5.3 未来方向

提升输入质量有几个可能的路径。基于深度信息的自适应裁剪可以改善物体裁剪的质量，多视角描述融合能滤除单帧的噪声和幻觉，换用更强的视觉编码器也可能直接提升 LLaaVA 的原始输出。An 等人^[5]提出的将深度信息与 RGB 特征联合编码的方法是一个可直接参考的技术方向。

真值验证链路的全自动化值得继续推进。把 Isaac Sim 渲染到 CG 管线标注到真值对比三个环节串联起来，可以直接在物体检测数量、类别准确率、三维位置误差和关系准确率四个指标上给出精确的量化结果，使“标注精度可验证”这一主张从概念变为可操作的评估流程。

下游训练验证是检验数据质量的终极方式。将 2395 条 QA 用于微调 Qwen2.5-VL 或类似的视觉语言模型，在 MathVista、MMMU、GQA 等标准视觉问答基准上评估微调前后的性能变化。GRPO 强化学习策略在 Code2Logic 的原始实验中使感知和推理错误率分别降低了 19.2% 和 11.5%^[2]，可将这一训练策略引入本文的数据训练流程。

扩展场景覆盖范围。ScanNet 和 HM3D 等更大规模的室内数据集是真实场景方面最直接的扩展选项。Isaac Sim 中可以构建更多样化的房型模板，引入包

含移动物体和时序变化的动态场景。Song 等人^[1] 关于具身代理在动态环境中多步规划的研究，为动态场景的评估范式提供了参考。

参考文献

- [1] SONG C H, WU J, WASHINGTON C, et al. LLM-planner: few-shot grounded planning for embodied agents with large language models[A/OL]. arXiv (2023). <https://arxiv.org/abs/2212.04088>.
- [2] TONG J, TANG J, LI H, et al. Code2Logic: game-code-driven data synthesis for enhancing VLMs general reasoning[J]. Advances in Neural Information Processing Systems, 2025, 38.
- [3] LIU H, LI C, WU Q, et al. Visual instruction tuning[A/OL]. arXiv (2023). <https://arxiv.org/abs/2304.08485>.
- [4] TEAM Q. Qwen2.5: a party of foundation models[EB/OL]. 2024. <https://qwenlm.github.io/blog/qwen2.5/>.
- [5] AN Z, SUN G, LIU Y, et al. Multimodality helps few-shot 3D point cloud semantic segmentation[M]//Proceedings of the International Conference on Learning Representations (ICLR). 2025.
- [6] ZHANG Z, ZHANG A, LI M, et al. Automatic chain of thought prompting in large language models[A/OL]. arXiv (2022). <https://arxiv.org/abs/2210.03493>.
- [7] CHENG X, et al. Revisiting chain-of-thought prompting: zero-shot can be stronger than few-shot[A/OL]. arXiv (2025). <https://arxiv.org/abs/2506.14641>.
- [8] ZHOU X, LI R, LIANG P, et al. Using LLMs in generating design rationale for software architecture decisions[A/OL]. arXiv (2024). <https://arxiv.org/abs/2504.20781>.
- [9] GU Q, KUWAJERWALA A, MORIN S, et al. ConceptGraphs: open-vocabulary 3D scene graphs for perception and planning[C]//IEEE International Conference on Robotics and Automation (ICRA). 2024.
- [10] JATAVALLABHULA K M, et al. gradSLAM: automagically differentiable SLAM[A/OL]. arXiv (2020). <https://arxiv.org/abs/1910.10672>.
- [11] REN T, LIU S, ZENG A, et al. Grounded SAM: assembling open-world models for diverse visual tasks[A/OL]. arXiv (2024). <https://arxiv.org/abs/2401.14159>.

- [12] STRAUB J, WHELAN T, MA L, et al. The replica dataset: a digital replica of indoor spaces[A/OL]. arXiv (2019). <https://arxiv.org/abs/1906.05797>.
- [13] NVIDIA Corporation. NVIDIA isaac sim 4.5[EB/OL]. 2025. <https://developer.nvidia.com/isaac-sim>.

致 谢

本论文的完成离不开许多人的帮助与支持。

首先，衷心感谢我的指导教师戈维峰副教授。在整个毕业设计过程中，戈老师从选题方向、研究方法到论文撰写规范都给予了悉心指导和宝贵建议。

感谢计算与智能创新学院的各位老师，在四年本科学习期间传授的专业知识和科研方法，为本论文的研究工作奠定了坚实的基础。

感谢开源社区的贡献者。本文的管线构建依赖于 `GradSLAM`、`Grounded-SAM`、`LLaVA`、`llama.cpp`、`Ollama` 等优秀的开源项目，`Code2Logic` 论文作者公开的 `GameQA` 数据集为本研究提供了重要的方法论参考和对比基线。`NVIDIA Isaac Sim` 团队提供了功能完备的仿真平台。

感谢我的同学和朋友们，在毕设过程中与我讨论技术问题、分享经验心得，在遇到困难时给予鼓励与支持。

最后，感谢我的家人多年来的关怀与付出，你们的理解与支持是我不断前行的动力。

完整 Prompt 模板

以下为本文在描述优化和关系推理阶段使用的完整 prompt 文本。

A.1 描述优化 System Prompt

You analyze object crops from an indoor 3D scene.

For each item, identify the object and output a concise description.

Output ONLY a JSON array:

```
[{"id": <number>, "class": "<category>", "caption": "<description>"}]
```

RULES for caption:

- Name the object category (door, floor, ceiling, window, table, chair, sofa, cabinet, shelf, light, curtain, pillow, rug, etc.)
- Describe material/color/shape (e.g. "wooden", "white", "rectangular")
- State its position if evident (e.g. "against the wall", "in the corner")
- Keep it to ONE sentence, under 20 words

FORBIDDEN - never use these words:

light, lighting, shadow, bright, dark, depth, atmosphere,
airy, mood, perspective, contrast, illuminate

If the input text is too vague to identify the object,
set class="unknown" and caption="An unidentifiable object."

If the text is clearly truncated/incomplete, skip the item entirely.

A.2 关系推理 Prompt

You are given a list of objects with their positions in a room.

Determine spatial relationships between pairs that are clearly related.

Output ONLY a JSON array:

```
[{"from": <id1>, "to": <id2>, "relation": "<type>", "reason": "<brief>"}]
```

Relation types: "next to" | "above" | "below" | "behind" |
"in front of" | "on"

Only include pairs that have a clear spatial relationship. Do not guess.

QA 数据 JSON Schema

B.1 Target Perception（目标感知）

```
{  
  "data_id": "<scene>-tp-<id:04d>",  
  "qa_type": "Target Perception",  
  "question": "What is object <id> in this scene?",  
  "answer": "<refined caption>"  
}
```

B.2 Spatial Reasoning（空间推理）

```
{  
  "data_id": "<scene>-sp-<src:04d>-<dst:04d>",  
  "qa_type": "Spatial Reasoning",  
  "question": "Where is object <dst> relative to object <src>?",  
  "answer": "Object <dst> is <relation> object <src>."  
}
```

B.3 Scene Comprehension（场景理解）

```
{  
  "data_id": "<scene>-sc",  
  "qa_type": "Scene Comprehension",  
  "question": "Describe the overall layout of this scene.",  
  "answer": "A <room_type> with <N> objects: <summary>..."  
}
```

学位论文独创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。论文中除特别标注的内容外，不包含任何其他个人或机构已经发表或撰写过的研究成果。对本研究做出重要贡献的个人和集体，均已在论文中作了明确的声明并表示了谢意。本声明的法律结果由本人承担。

作者签名：

任立德

日 期：